TASK 3 – DBMS
Seller and Inventory Management System

1. Aim

To design and implement a Seller and Inventory Management System using MySQL to manage sellers, products, stock, and inventory status.

2. Objectives

- Create Seller and Inventory tables.
- Store seller and product information.
- Establish relationships between sellers and products.
- Track available and unavailable products.
- Maintain stock quantities.
- Generate inventory status reports.

3. Database Design

The system contains three main tables:

Seller

- Seller ID
- Seller Name
- Phone
- Email

Product

- Product ID
- Product Name
- Price
- Seller ID

Inventory

- Inventory ID
- Product ID
- Stock Quantity

• Product Status

Relationship:

Seller → Product → Inventory

One seller can have many products, and each product has its inventory details.

4. MySQL Code – Create Database

```
CREATE DATABASE seller_inventory;
USE seller_inventory;
```

5. Create Seller Table

```
CREATE TABLE Seller (
    seller_id INT PRIMARY KEY,
    seller_name VARCHAR(50),
    phone VARCHAR(15),
    email VARCHAR(50)
);
```

6. Create Product Table

```
CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    price DECIMAL(10,2),
    seller_id INT,
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)
);
```

7. Create Inventory Table

```
CREATE TABLE Inventory (
    inventory_id INT PRIMARY KEY,
    product_id INT,
    stock_quantity INT,
    status VARCHAR(20),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

8. Insert Seller Data

```
INSERT INTO Seller VALUES
(1, 'Ravi Stores', '9876543210', 'ravi@gmail.com'),
(2, 'Kumar Traders', '9876501234', 'kumar@gmail.com'),
(3, 'Anu Shop', '9876512345', 'anu@gmail.com');
```

9. Insert Product Data

```
INSERT INTO Product VALUES
(101, 'Laptop', 55000.00, 1),
(102, 'Mouse', 500.00, 2),
(103, 'Keyboard', 1200.00, 3),
(104, 'Monitor', 10000.00, 1);
```

10. Insert Inventory Data

```
INSERT INTO Inventory VALUES
(1, 101, 10, 'Available'),
(2, 102, 25, 'Available'),
(3, 103, 0, 'Unavailable'),
(4, 104, 5, 'Available');
```

11. Display Seller Details

```
SELECT * FROM Seller;
```

12. Display Product Details

```
SELECT * FROM Product;
```

13. Display Inventory Details

```
SELECT * FROM Inventory;
```

14. Display Seller and Product Information

```
SELECT
    Seller.seller_name,
    Product.product_name,
    Product.price
FROM Seller
JOIN Product
ON Seller.seller_id = Product.seller_id;
```

15. Check Available Products

```
SELECT
    Product.product_name,
    Inventory.stock_quantity
FROM Product
JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.status = 'Available';
```

16. Check Unavailable Products

```
SELECT
    Product.product_name,
    Inventory.stock_quantity
FROM Product
JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.status = 'Unavailable';
```

17. Inventory Status Report

```
SELECT
    Product.product_name,
    Inventory.stock_quantity,
    Inventory.status
FROM Product
JOIN Inventory
ON Product.product_id = Inventory.product_id;
```

18. Update Stock

```
UPDATE Inventory
SET stock_quantity = 15,
    status = 'Available'
WHERE product_id = 101;
```

19. Delete a Product

```
DELETE FROM Inventory
WHERE product_id = 104;

DELETE FROM Product
WHERE product_id = 104;
```

20. Sample Output

Product	Stock	Status
Laptop	10	Available
Mouse	25	Available
Keyboard	0	Unavailable
Monitor	5	Available

21. Conclusion

The Seller and Inventory Management System successfully manages seller details, products, and stock information. The relationships between the tables help maintain organized data. The system can identify available and unavailable products and generate inventory status reports efficiently.